

AIM- To configure a GPIO pin of STM32F446RE as digital output and verify LED blinking operation using software delay routines.

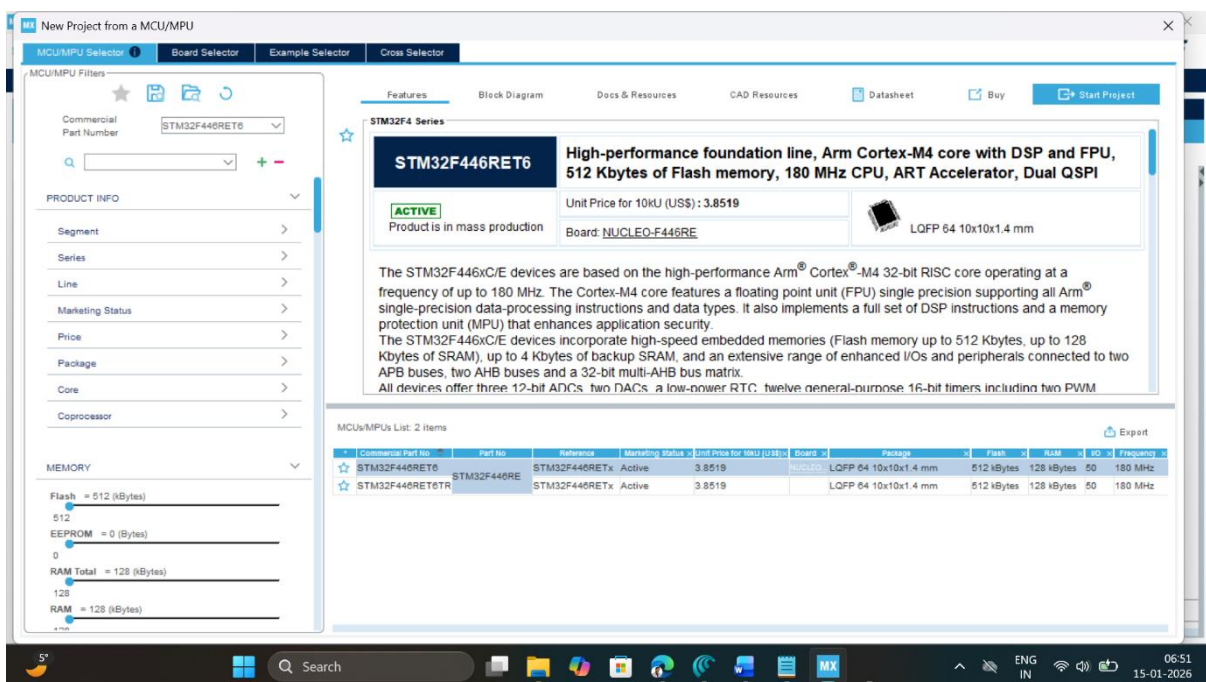
APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

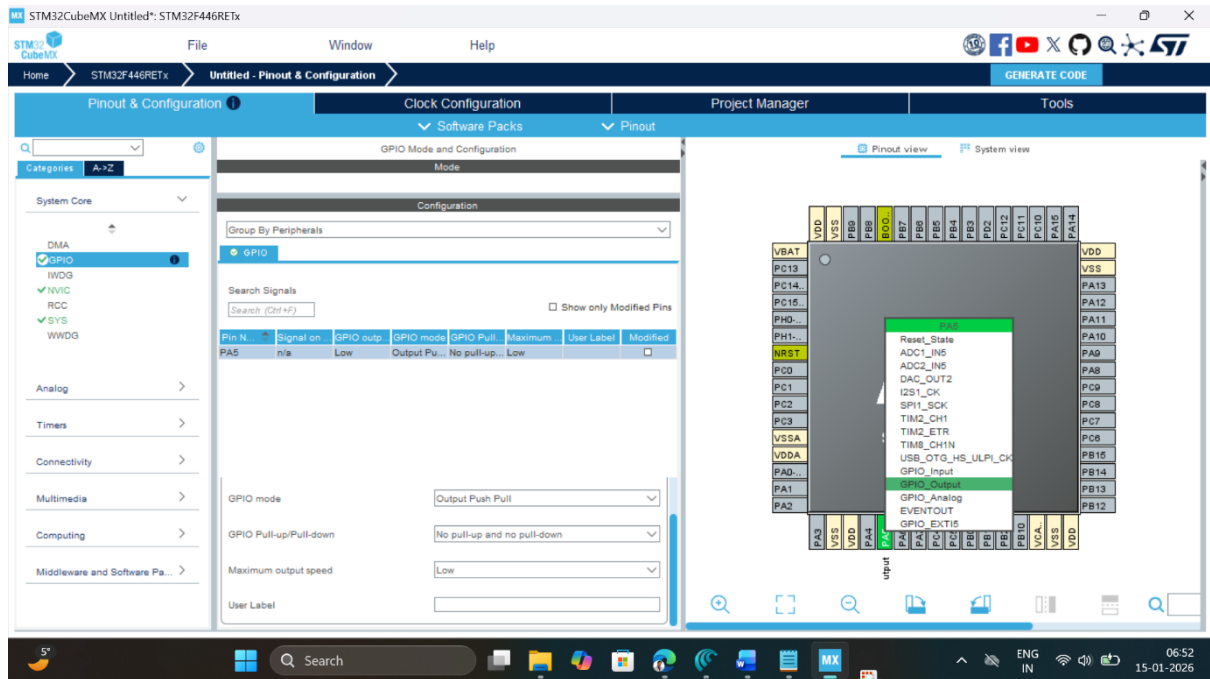
STM32F446RE provides multiple general-purpose I/O (GPIO) ports (GPIOA–GPIOH) that can be configured as input, output, alternate function, or analog through GPIO control registers or HAL. On the Nucleo-F446RE board, the green user LED **LD2** is connected to Arduino D13, which corresponds to microcontroller pin **PA5** on most board revisions. When PA5 is configured as a **push-pull output**, the pin can be driven to logic high (near 3.3 V) or logic low (0 V), which in turn turns the LED on and off through an on-board current-limiting. Using STM32CubeIDE and HAL, functions like HAL_GPIO_TogglePin() and HAL_Delay() can be used inside an infinite loop to generate a periodic LED blink without directly programming registers.

PROCEDURE

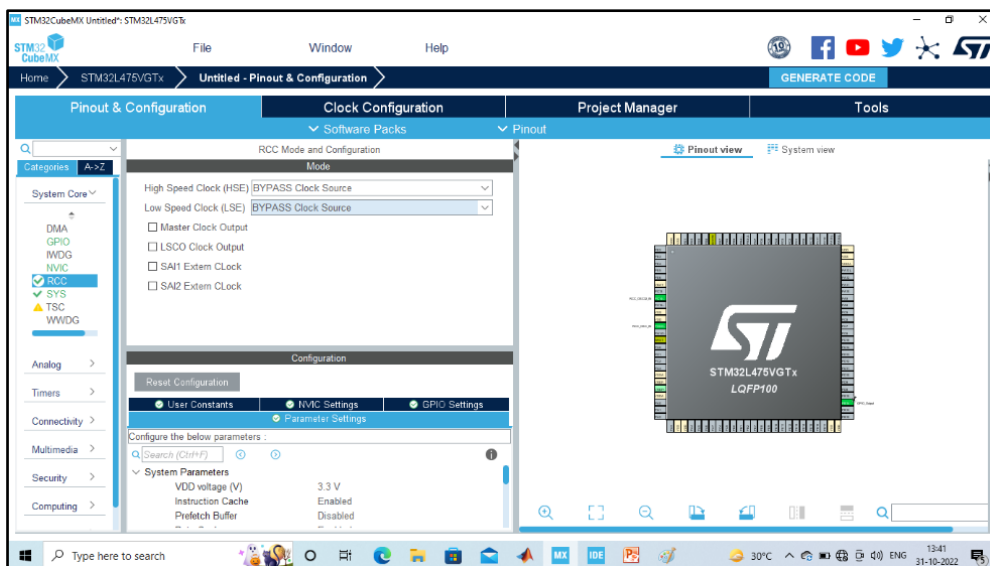
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



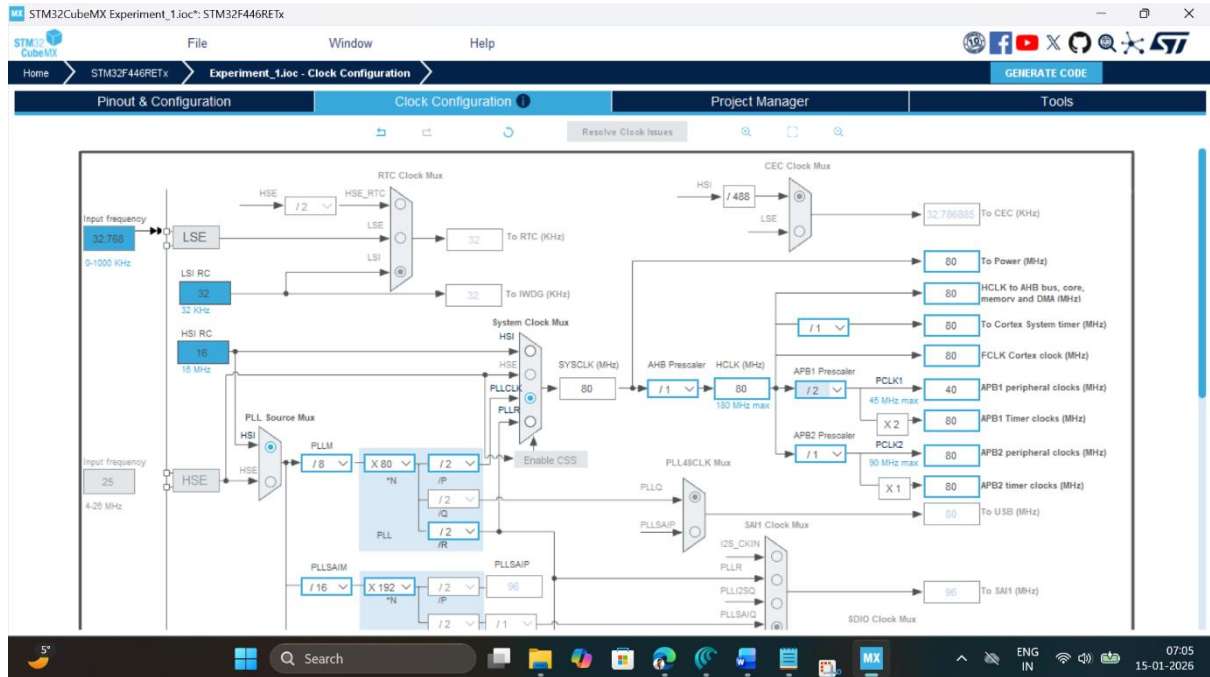
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) relates to it in the board.



4. **Clock Selection:** Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency.



6. Write Project name and select IDE.

7. Click on generate code.

8. Click on open project.

9. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.

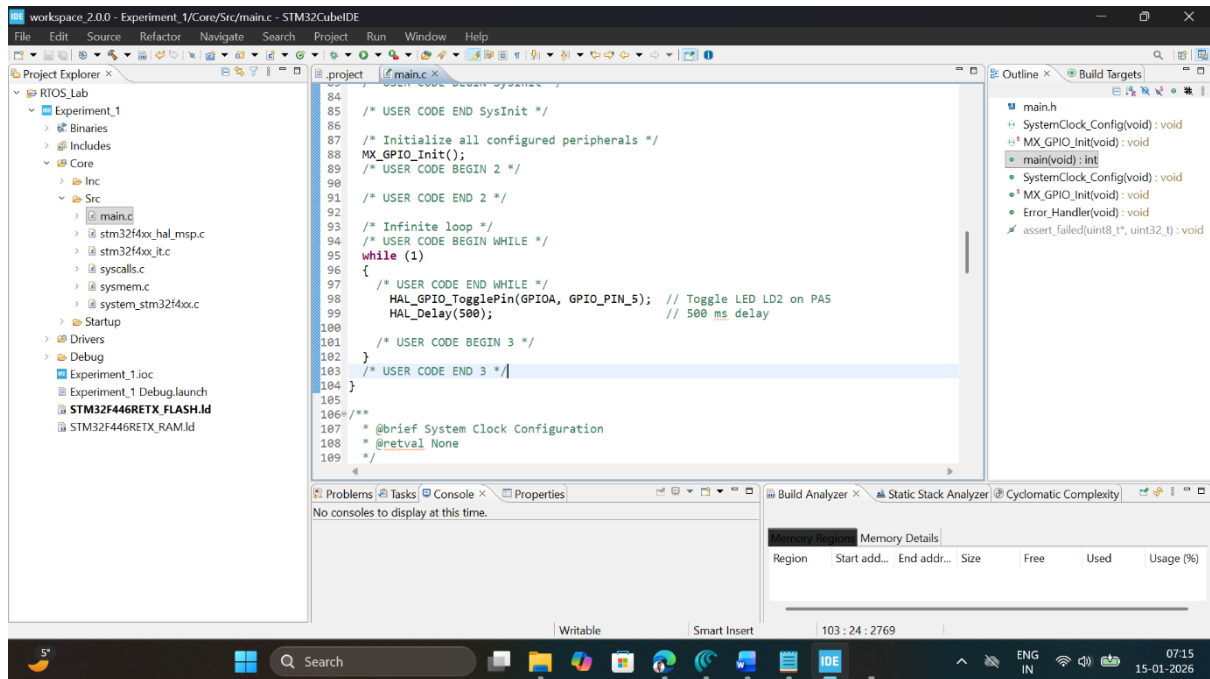
10. Open the main source code on Cube IDE by clicking on **main.c**

11. To generate bin file and hex file, right click on project and click properties.

12. Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close".

13. After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
```



14. To blink the connected LED on the board, connect the board, click on build, click 'ok' and click 'switch' on popups.

15. Now you are in debugging mode, click 'Resume' to execute the program.

OBSERVATION: Change the delay value (e.g., 200 ms, 1000 ms), re-build, re-run, and note the change in blink speed and fill the observation table.

S. No.	GPIO pin used	Mode / Type / Pull	Delay value (ms)	Approx. blink period (s)	Comment (slow/fast/visible)
1	PA5	Output, PP, No PU/PD	100	0.2 s	Very fast, difficult to notice
1	PA5	Output, PP, No PU/PD	200	0.4 s	Fast but visible
2	PA5	Output, PP, No PU/PD	500	1.0 s	Normal, clearly visible
3	PA5	Output, PP, No PU/PD	1000	0.2 s	Slow, very clearly visible

Result:

The GPIO pin PA5 of STM32F446RE on the Nucleo-F446RE board was configured as a digital output in STM32CubeIDE, and the on-board user LED LD2 was successfully made to blink using software delay routines implemented with HAL functions

Reflection summary:

Through this experiment, I gained a clear and practical understanding of GPIO configuration parameters of STM32 microcontrollers. GPIO (General Purpose Input Output) pins are highly flexible and can be configured in different modes depending on the application requirement. The main GPIO parameters include **pin mode**, **output type**, **pull-up/pull-down selection**, and **speed**. By configuring PA5 as a digital output for LED blinking, I learned how these parameters directly affect the behavior of the pin.

The **mode selection** determines whether the pin works as input, output, alternate function, or analog. In this experiment, selecting output mode allowed the microcontroller to drive the LED. The **output type** (push-pull or open-drain) controls how the pin drives voltage. Push-pull mode allows the pin to actively drive both high and low logic levels, making it suitable for LED control. The **pull-up and pull-down resistors** are used to avoid floating states when a pin is configured as input; since PA5 was used as output, no pull-up or pull-down was required.

Using STM32CubeMX helped me understand that GPIO configuration is mostly done graphically and internally mapped to register settings by HAL. This reduces the complexity of direct register programming and makes development faster and less error-prone. I also understood the importance of correct clock configuration because GPIO peripherals require clock enable before operation.

By changing delay values, I observed how software timing affects output behavior, which helped relate code execution time to physical output signals. Overall, this experiment strengthened my understanding of how STM32 GPIO pins are configured and controlled, and how software parameters translate into real hardware actions. This knowledge is essential for interfacing LEDs, switches, sensors, and communication peripherals in embedded systems.